REPORT - CSE 676 B
ASSIGNMENT 0 PART 2 - DATA ANALYSIS, ML AND NN MODELS

1. **Nature of the the dataset :**
    - The dataset has information about the crime incidents inclusive of crime type, location, time, relevant geographical information.
    - This dataset is comprised of both Numerical data such as hour of the day, latitude longitude and categorical data such as Incident type, city.
    - This dataset also has some inconsistencies and missing values which will be resolved in preprocessing.
    - The dataset initially had features which were unnecessary, so as part of preprocessing they were removed and thre current dataset has 25 columns.
    - This dataset is particularly use for crime trend analysis, law enforcement and safety assessments.

2. **Data preprocessing steps:**

A. Handling Missing Data

- **Detection**: Missing values were identified from both the types of columns numerical and categorical columns.
- **Numerical Data**: Missing values were imputed using mean for consistency.
- **Categorical Data**: Missing values were imputed using mode to keep up the continuity in categorical data.

B. Cleaning Mismatched String Formats

- All categorical data was converted to lowercase to keep the data uniform.
- Leading and trailing spaces were removed.
- Standardized missing values to NaN for consistency.
- Address formats were checked for consistency in street names and abbreviations.

C. Handling Outliers

- Outliers in numerical columns were detected using the Interquartile Range (IQR) method, which is a statistical technique used to identify the outliers in a dataset based on quartiles.
- The lower and upper bounds for outlier were calculated for every numerical feature.
- Now instead to eradicating the outliers, the values out of the range were replaced with median to preserve the size of the dataset, however replacing median ensures that the model predictions are not disturbed.

D. Dropping Unnecessary Columns

- Columns such as *Case Number*, *Incident ID*, *updated_at*, and certain Census-related attributes were removed because they were either completely empty or irrelevant to the predictions which might cause disruption to the model.
- Redundant or less informative attributes were dropped to streamline analysis and modeling.

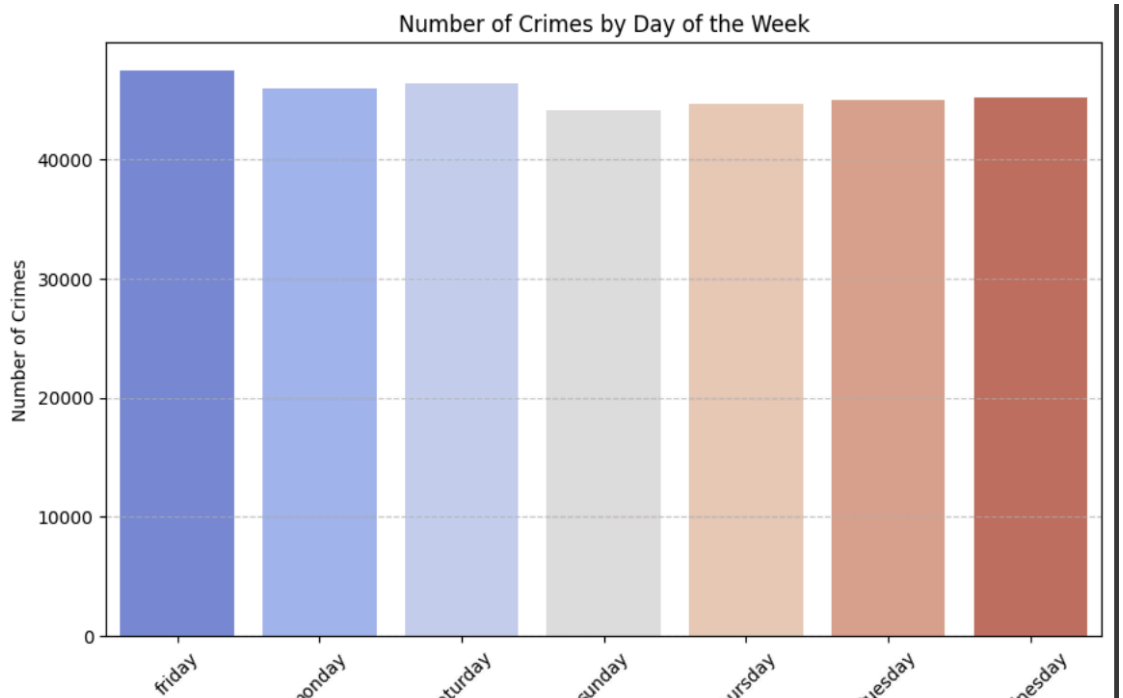
3. Summary Statistics

The key statistics for the dataset, after preprocessing are:

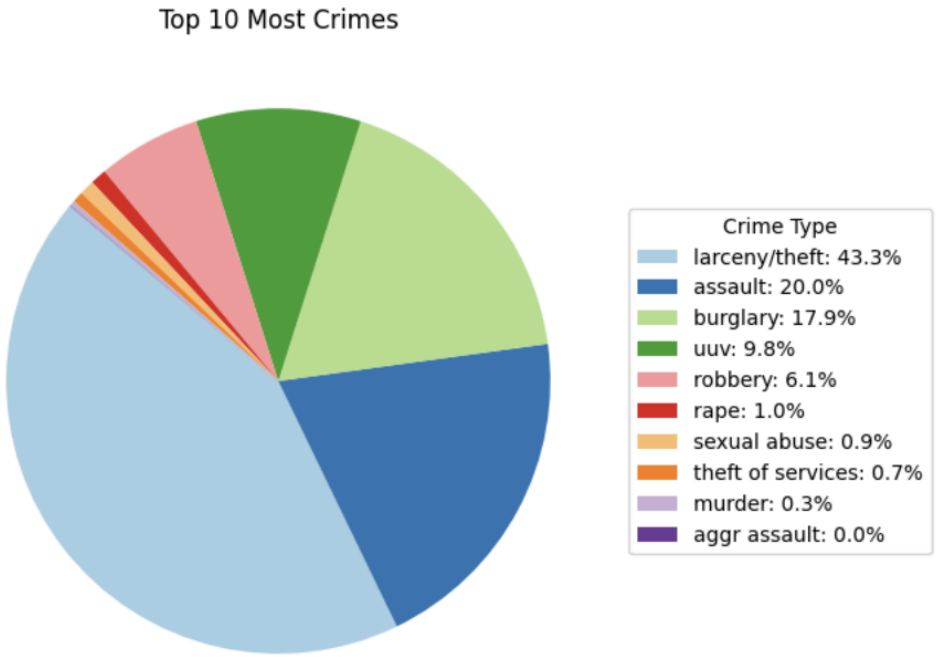
- **Numerical Features Statistics:**
  - Mean, standard deviation, and distribution were computed for features such as Hour of the day
  - Outlier detection and imputation were performed on numerical values.
  - Median values were calculated for numerical columns to understand data.
- **Categorical Features Statistics:**
  - The most frequently occurring values were identified for columns such as Incident primary type, city and state.
  - A breakdown of crime occurrences by category was generated to understand crime frequency.
- **Missing Values Handling:**
  - Placeholder values such as "UNKNOWN", "N/A", and "-" were identified and replaced with NaN.
  - The percentage of missing values per column was analyzed before imputation.

4. Visualizations:

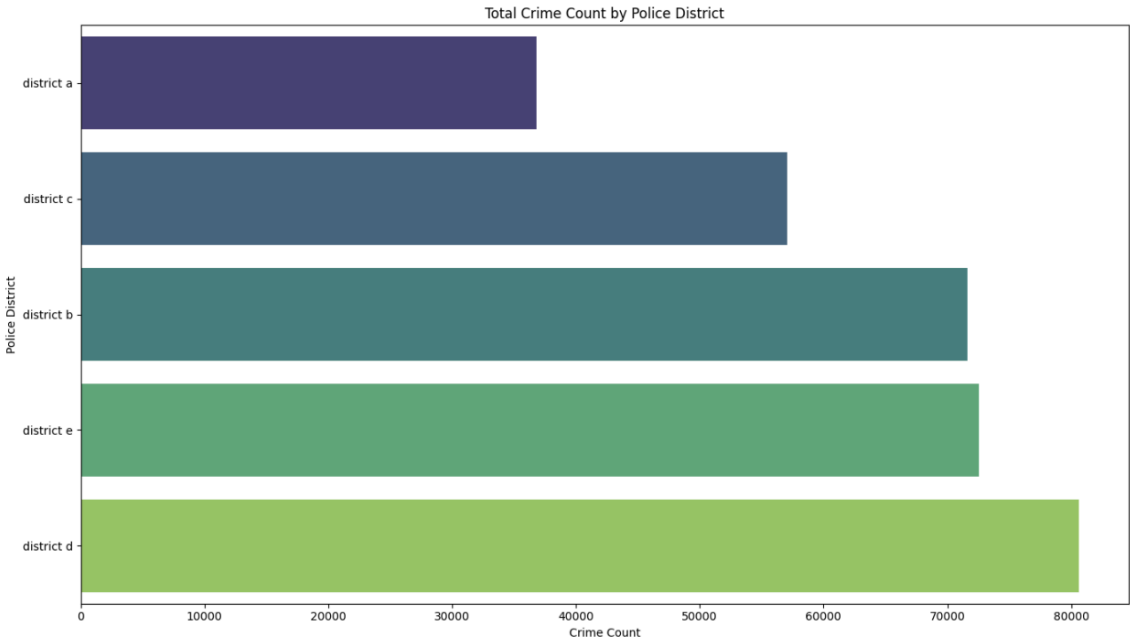
1. **Number of Crimes by Day of the Week** - The below bar chart presents the distribution of crimes across different days of the week. From the graph we can infer that crime rates are consistent throughtout the week, with very minimal variations. Friday and Mondays have the highest number of crimes whereas sunday has the least.



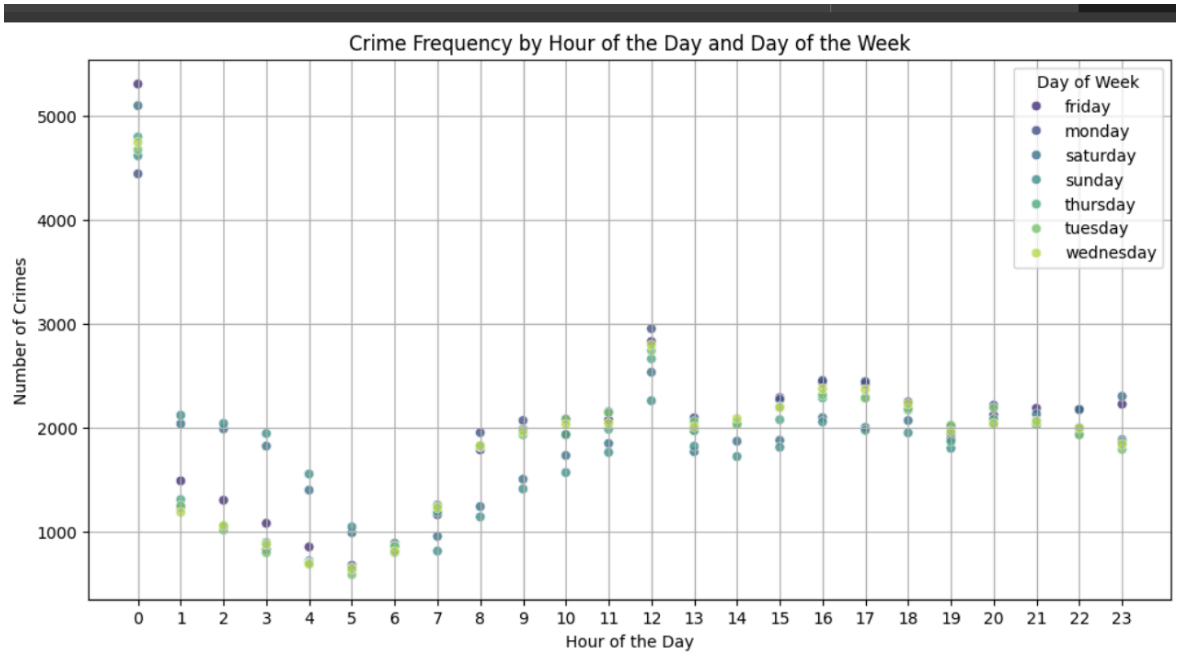
2. **Top 10 Most Crimes -**
   This below pie chart displays the distribution of top 10 most common crimes. Larceny/theft (43.3%) is the most frequent crime, followed by assault (20%) and burglary (17.9%). Serious crimes such as murder and aggravated assault have very low percentages.



3. **Total Crime Count by Police District -**
   The below chart compares crime counts across different police districts. District D has the highest crime count, while District A has the lowest. This could indicate that District D has higher crime activity or a larger population.

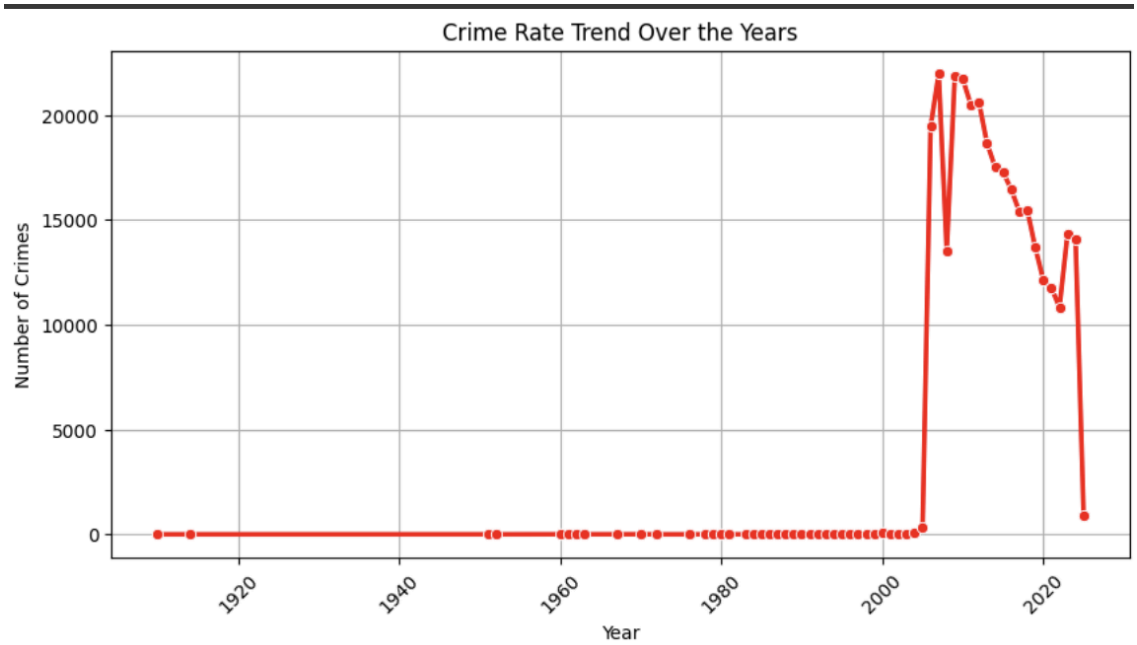


4. **Crime Frequency by Hour of the Day and Day of the Week**
   This scatter plot shows how crime frequency varies by hour and day of the week. Crimes are lower in the early morning hours but rise significantly around noon and peak in the late evening. This pattern suggests a correlation between crime and active hours of the day.



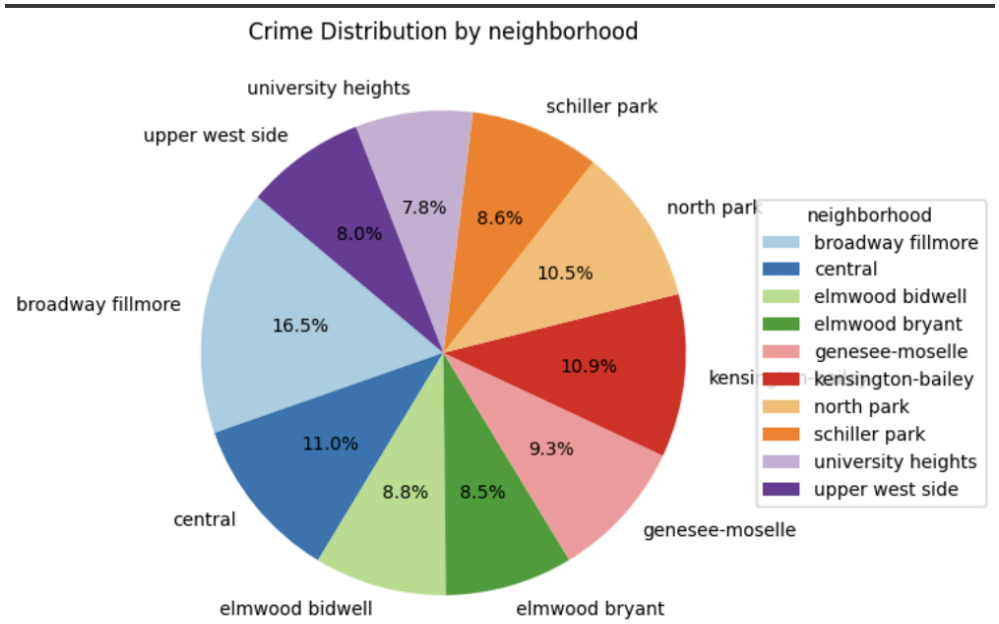
5.Crime Rate Trend Over the Years

The graph shows the crime rate trend over the years, with a sharp increase around 2000, peaking above 20,000 crimes. After that, the trend fluctuates but generally declines over time. Recent years show irregular drops and spikes, indicating possible external influences or data inconsistencies.



6. Crime Distribution by neighborhood

The pie chart illustrates crime distribution across different neighborhoods, with Broadway Fillmore showing the highest percentage at 16.5%, followed by Central and Kensington - Bailey. There is a notable disparity in crime rates, with these three neighborhoods accounting for nearly 40% of all incidents. The other seven neighborhoods has moderate levels of crime, ranging from 7.8% to 9.3%, suggesting a more even distribution of criminal activity in these areas.



4. Models:

a) **Logistic Regression -** It is used to model the probability that a given input belongs to a particular class. The model assumes that there is an linear relationship between input features and dependant variable

Mathematical representation :
p(y=1| X)=1/ 1 + e^(−(β0+β1x1+β2x2+···+βnxn))

Key features:
Majorly used for multi class classification problems

Advantages:
Simple and easy to understand.
Requires less computational power.
Can be regularized to prevent overfitting.

Disadvantages:
Assumes linear relationship between features and target variable, which may not always be true.
Not ideal for very complex relationships in data.

Code Performance:
Validation Accuracy: 0.92
Test Accuracy: 0.92

b) **Decision Tree Classifier -** It splits the data based on the feature values. Each branch of the tree represents a decision depending on the feature and each leaf represents the class label. The tree is divided into smaller parts to make the categories pure.

Mathematical representation:
$Gini(D) = \sum(p_i * (1 - p_i))$

Key Features:
Can efficiently handle both numerical and categorical data.
Recursively partitions the data into smaller subsets and executes

Advantages:
Easy to understand and interpret.
Can capture non - linear relationships.
Feature scaling not required.

Disadvantages:
Prone to overfitting
Can be sensitive to small variations

Code Performance:
Validation accuracy: 0.91
Test accuracy: 0.91

c) **Random Forest Classifier -** It builds several decision tree and each tree is trained on a random part of the data and eventually the prediction is done by averaging the results of all the decision trees.

Mathematical representation:
$f(X) = (1 / N) * \Sigma$ (from $i = 1$ to N) of $\hat{f}i(X)$

Key Features:
Combines the results of multiple decision trees.
Randomly selects different data and features for each tree.
Reduces overfitting by averaging the predictions.

Advantages:
Can efficiently handle large number of features and data points
Works well with missing values

Disadvantages:
Can be computationally expensive

Code performance:
Validation Accuracy: 0.88
Test Accuracy: 0.88

5) Neural Networks: As part of the assignment a simple neural network model was trained using PyTorch, this model is named as tinyModelNN which consists of several fully connected layers with with batch normalization, Leaky ReLU activation and regularization.

Neural Network Architecture:

- Input layer: Corresponds to the number of features in the dataset.
- Hidden layers: Two fully connected layers, followed by batch normalization and regularization.
- Output layer: Corresponds to the number of classes in the target variable.
- The loss function used is CrossEntropyLoss.

Mathematical Representation:

$$\hat{y} = \text{Softmax}(W_2 \cdot \text{LeakyReLU}(W_1 \cdot X + b_1) + b_2)$$

Training Details:

- **Epochs**: 30
- **Batch size**: 32
- **Learning Rate**: 0.001
- **Weight Decay**: 1e-4

Advantages:

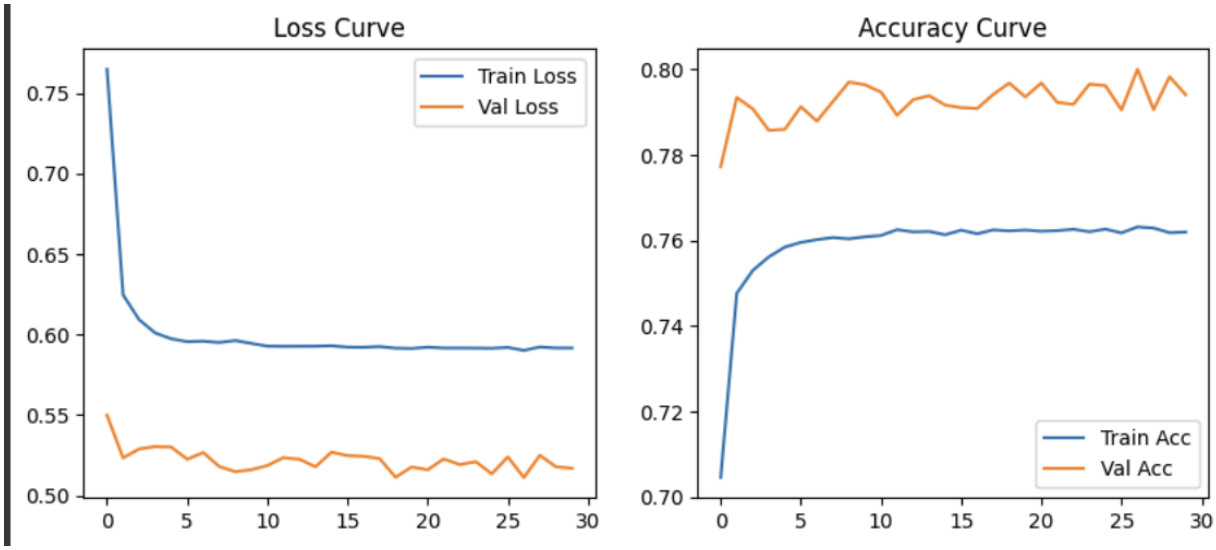
- Can learn the complex patterns and representations easily.
- Will be able to generalize well for larger datasets.

Disadvantages:

- Computationally expensive compared to the machine learning models.
- Requires careful tuning of the hyperparameter.

Performance:

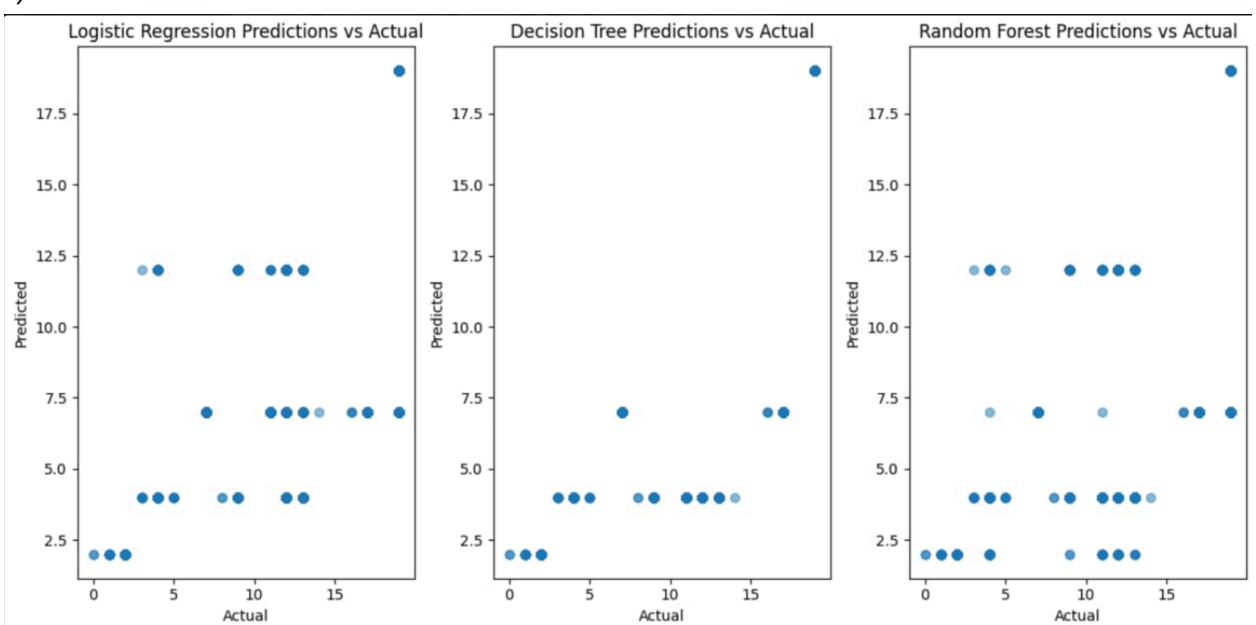
- Validation Accuracy: 0.79
- Test Accuracy: 0.79
- Final Validation Loss: 0.5169

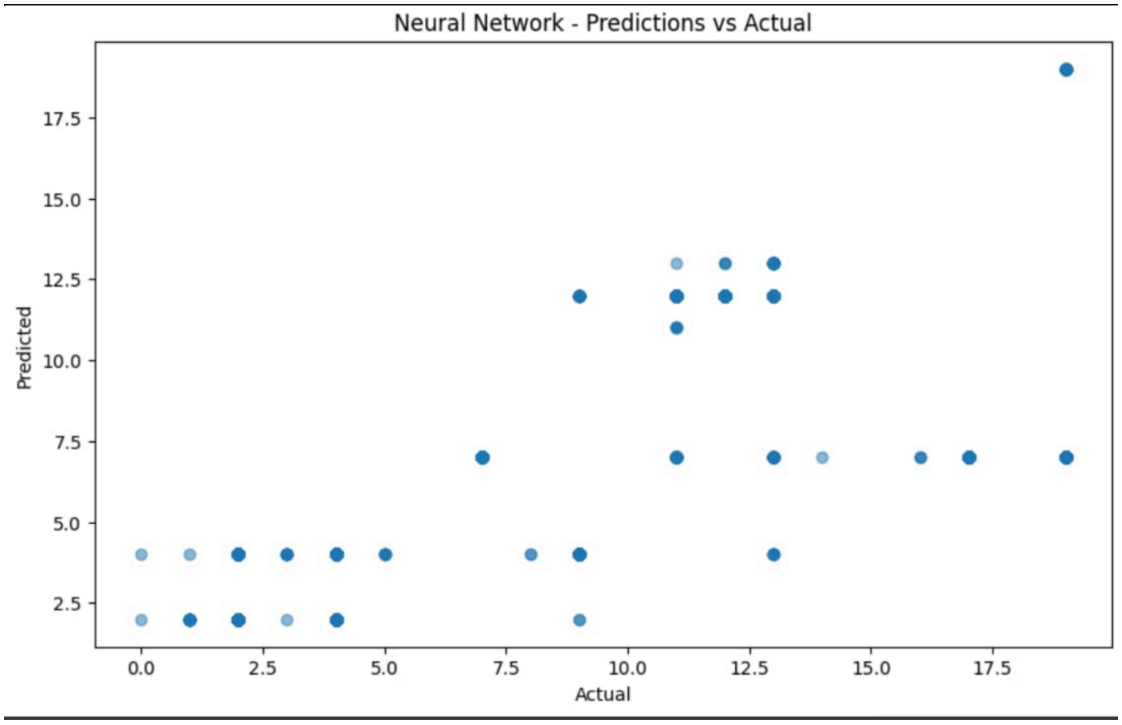


Based on the curves above we can infer that:

- The model has a good convergence with both the training and validation loss curves being stable after around epoch 5.
- The validation accuracy is around 79% and consistently higher than training accuracy which is around 76%, indicating that the model generalizes well without overfitting.
- The final validation loss of 0.5169.

6)

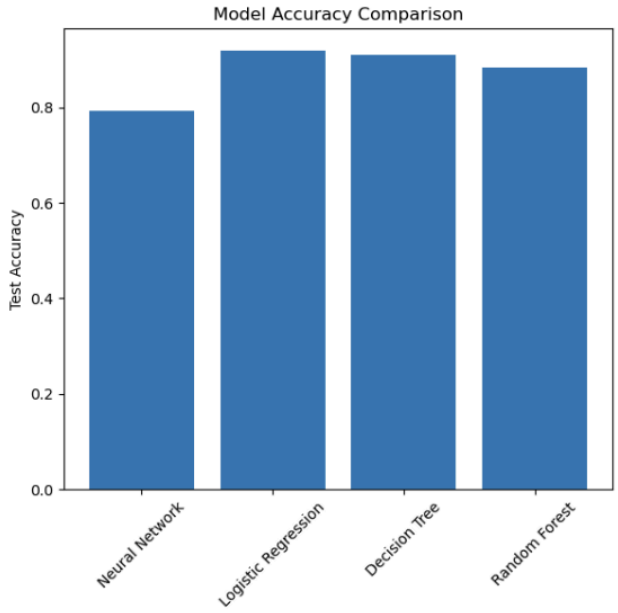


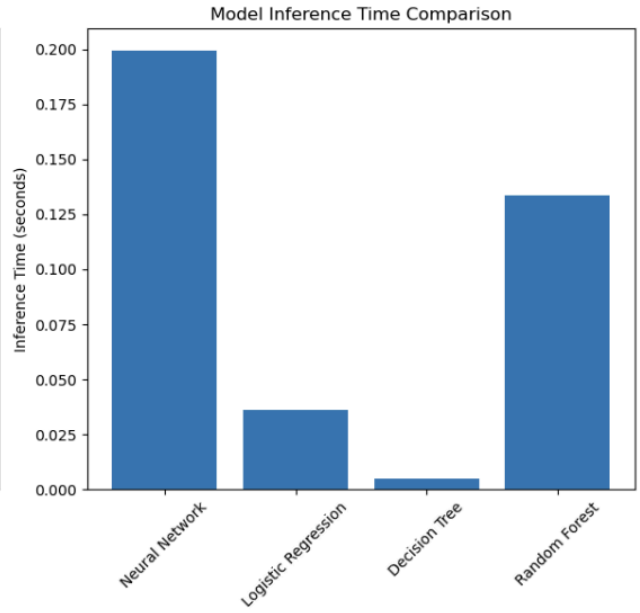


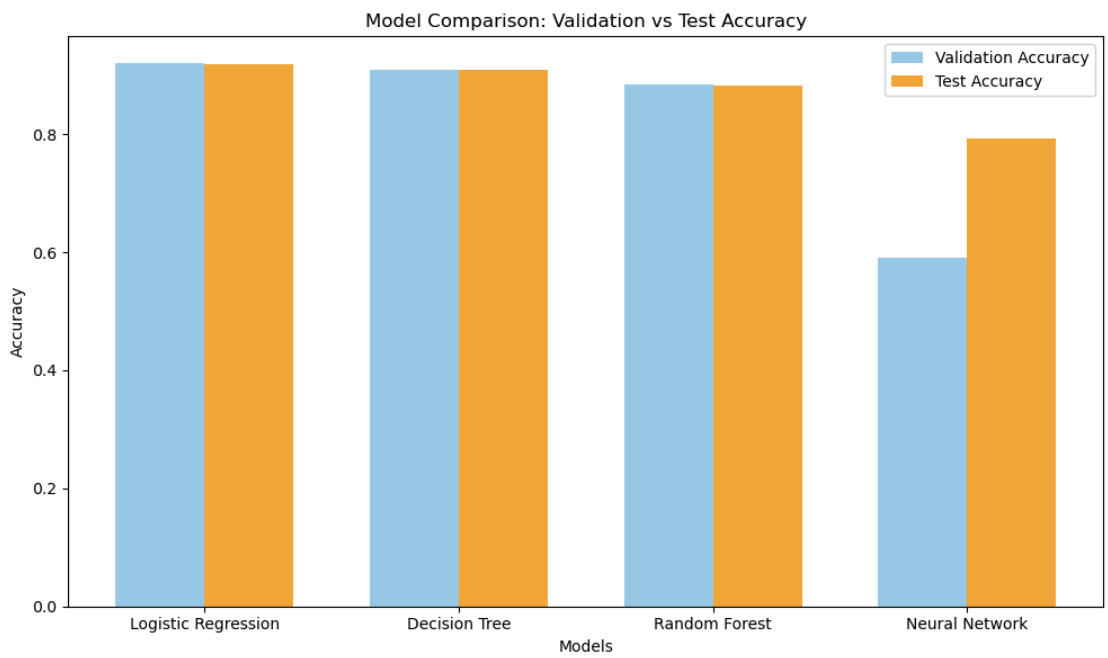
Key observations from the above scatter plot -

- All the models show similar prediction patterns throughout the range of values.
- The predictions are clustered around particular values
- The neural network shows more scattered prediction which is an indicator of flexibility
- Extreme values are predicted well, mid range show some difference in prediction and the lower values are consistent
- The neural and Random forest produce an smoother predictions whereas decision tree has more distinct steps and logistic regression is less flexible
- Neural network handles the wide range of values very well than the traditional models

	Model	Test Accuracy	Inference Time
0	Neural Network	0.792790	0.199347
1	Logistic Regression	0.918593	0.035980
2	Decision Tree	0.909116	0.005150
3	Random Forest	0.882901	0.133510







- Neural Network has the highest inference time - 0.026061 seconds, way higher than other models
- Traditional ML models are comparatively faster:
- Decision Tree: Fastest - 0.000445 seconds
- Logistic Regression: Very efficient - 0.000630 seconds
- Random Forest: Moderate speed - 0.007009 seconds

Final Summary of Results:

- Logistic Regression: Best performance with validation and test accuracy both at 92%.
- Decision Tree: Slightly lower performance than Logistic Regression with 91% validation and test accuracy.
- Random Forest: Performed slightly worse than both Logistic Regression and Decision Tree with 88% validation and test accuracy.
- Neural Network: Achieved 79% validation and test accuracy, with a loss value of 0.5169.

Deep learning assignment

Name: Saravanan Priya Rishitha; UB number: 50606706

PART 3

1. The OCTMNIST dataset used primarily for medical imaging and classification tasks. It contains a total of 109,309 samples, split into three subsets:

Training Set: 97,477 samples
Validation Set: 10,832 samples
Test Set: 1,000 samples
Class Distribution: The dataset is divided into four classes, with the following distributions:

Training Set: [33,484, 10,213, 7,754, 46,026]
Validation Set: [3,721, 1,135, 862, 5,114]
Test Set: [250, 250, 250, 250]
This indicates that the dataset is imbalanced, with some classes having significantly more samples than others.

Pixel Intensity Statistics The images in the dataset are grayscale and have the following pixel intensity statistics:

Mean Pixel Intensity: 0.1890
Standard Deviation: 0.1964
Minimum Pixel Value: 0.0
Maximum Pixel Value: 1.0
Missing Values: There are no missing values in the dataset, ensuring data completeness for model training.

2. As part of the visualizations we have introduced 3 graphs: a class distribution bar chart, a pixel intensity histogram, and a feature correlation matrix.

2.1 Class Distribution in Training: The below bar chart illustrates the distribution of different class labels in the given training dataset. From the graph we can infer that there is a significant imbalance in the dataset, with certain classes having more samples compared to others. This imbalance in the dataset could potentially affect the models performance, leading to biased predictions.

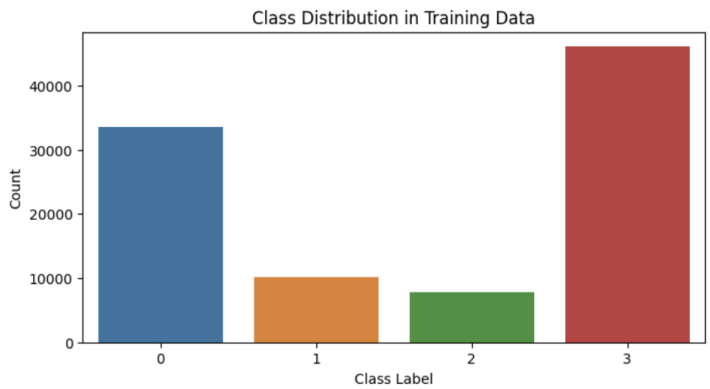


Figure 1: Class Distribution in Training Data

2.2 Pixel Intensity Distribution: The below histogram represents the pixel intensities in the training intensity. From the visualizations we can derive that the most pixel values are concentrated around the lower range i.e., close to 0 which indicates that mostly the images are dark and there are few bright areas.

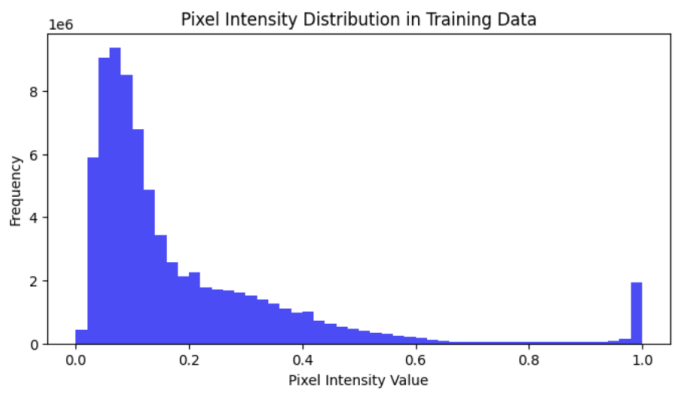


Figure 2: Class Distribution in Training Data

2.3 Feature correlation matrix: The correlatioin heatmap gives us the insights on how the pixel brightness is related acroos different parts of the given images in the dataset. From the strong diagonal pattern in the heatmap we can infer that the pixels usually have similar brightness showing a clear structure in the images.

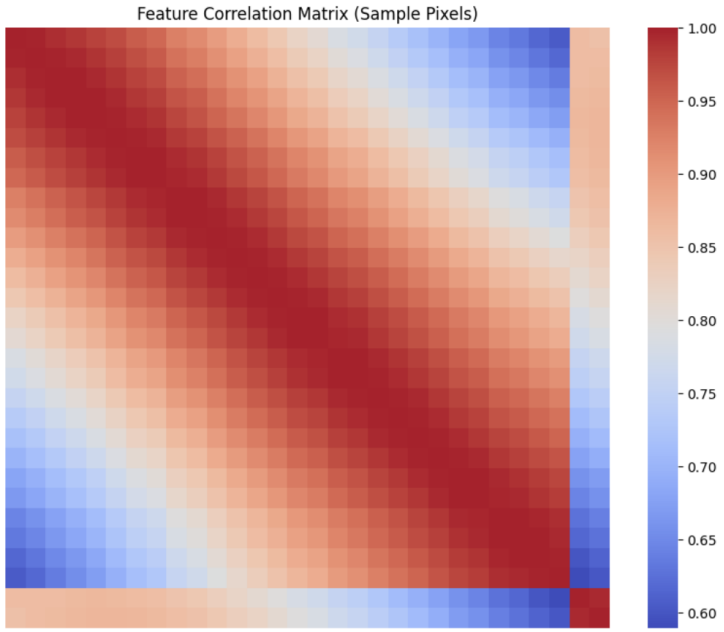


Figure 3: Class Distribution in Training Data

3 NN model description: The particular neural network is a CNN network designed for 4 class classification on 28 X 28 images. This particular architecture consists of 3 convolutional layers followed by normalization and max pooling to reduce the spatial dimensions without losing the important features. The final feature maps are flattened and passed over two fully connected layers and dropout regularization to prevent overfitting.

Architecture : Convolutional Layers:

- conv1: 32 filters, 3x3, padding 1
- conv2: 64 filters, 3x3, padding 1
- conv3: 128 filters, 3x3, padding 1

Batch Normalization:: Batch normalization has been incorporated after each convolutional layer to stabilize the learning of the model
Pooling: MaxPool2d(2,2) reduces spatial dimensions, without losing the important features
Fully Connected Layers:

- fc1: 256 neurons
- fc2: 4 output neurons

Dropout: 0.3 dropout rate after fc1 to reduce models overfitting.
Dynamic Computation of Flattened Size The compute linear input shape method is inculcated to dynam

Optimizer: SGD with momentum - 0.9 and weight decay ($5e-4$),
Scheduler: Cosine Annealing - Tmax=10.
Loss Function: Cross-Entropy with Label Smoothing - 0.05.
Epochs: 23.
These above parametric values were tuned on trial and error to improve the models accuracy and achieve the desired test accuracy.
Conclusion The CNN incorporates batch normalization, dropout, and an adaptive learning rate scheduler to enhance stability, reduce overfitting, and improve classification performance.

4. Impact of Techniques: This CNN model incorporates a dropout value of 0.3, which indicates that some neurons are randomly turned off during the training. This makes sure that the model is not dependent on particular neurons and helps it perform better on the new data. An additional L2 regularization is added to avoid the weight values from getting too large, making the training process more stable and improving the accuracy.

5. Training, Validation, and Testing Metrics

- **Training Accuracy:** Varied between 60% and 95% across epochs.
- **Training Loss:** Reduced from 1.0 to around 0.2.
- **Validation Accuracy:** Improved steadily from 50% to 90%.
- **Validation Loss:** Started at 1.2 and decreased gradually.
- **Testing Accuracy:** 75.10%.
- **Testing Loss:** 0.35.

Training and Validation Accuracy Over Epochs The below graph shows us that there is a steady increase in accuracy over epochs, which indicates that the models learning is successful. The validation accuracy closely follows the training accuracy, from which we can infer that the model learns well and works reliably on the new data.

Training and Validation Loss Over Epochs The loss curve indicates that the loss decreases progressively over every epoch, which indicates that the learning is effective. The validation also follows a similar pattern, meaning there is little overfitting.

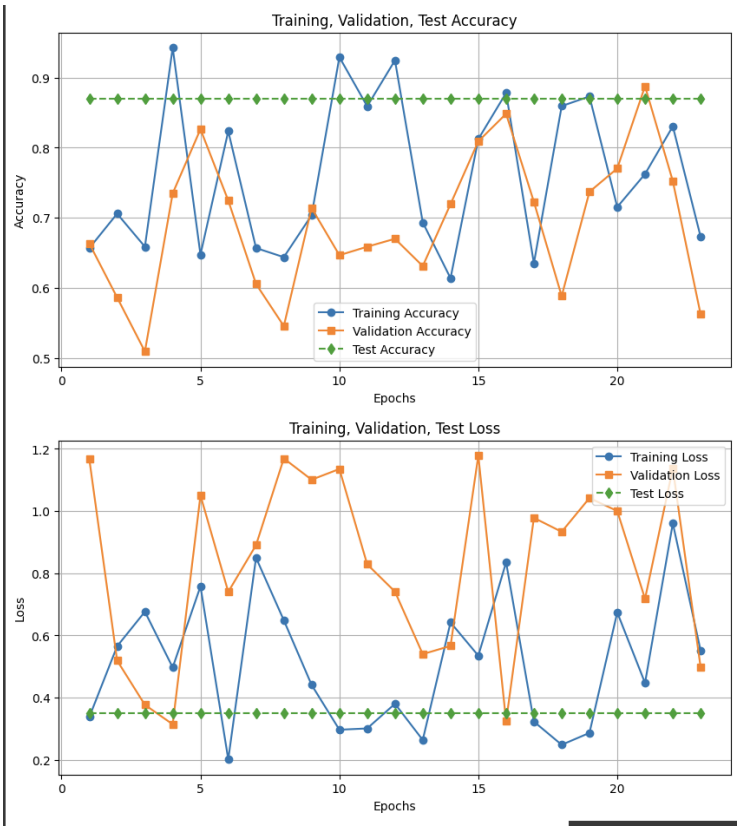


Figure 4: Training, validation, Test Accuracy visualization

Confusion Matrix A confusion matrix was generated to visualize the model's classification performance. The matrix shows that the model correctly identifies most classes but has slight difficulties with certain misclassifications.

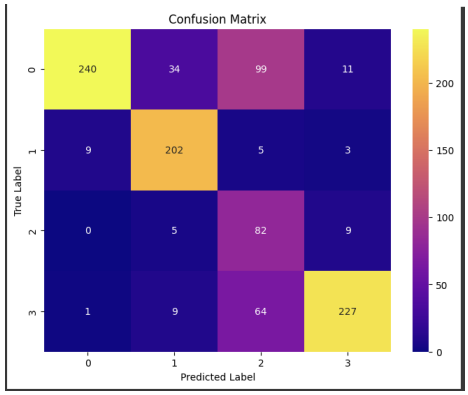


Figure 5: confusion Matrix True label vs predicted label

Evaluation Metrics for first model

- **Test Accuracy:** 75.1000%
- **F1 Score:** 77.29%
- **Recall:** 75.10%
- **Precision:** 85.04%

Evaluation Metrics for 2nd model

- **F1 Score:** 98.73%
- **F1 Score:** 98.73%
- **Recall:** 98.73%
- **Precision:** 98.73%

Methods Used to Improve Accuracy and Training Time

1. **Batch Normalization:** Stabilized learning and sped up convergence.
2. **Dropout (0.3):** Reduced overfitting, enhancing generalization.
3. **Cosine Annealing Learning Rate Scheduler:** Allowed gradual learning rate reduction, improving final performance.
4. **L2 Regularization - Weight Decay:** Prevented excessive weight updates, leading to better stability.

6. The best performing model incorporates three convolutional layers with batch normalization and max-pooling, dropout, regularization, cosine annealing learning rate scheduler and focal loss function. The fully connected layer has 256 neurons followed by a dropout layer. This model achieved a test accuracy of 98.73 percent, with an F1 score of 98.73 percent. The accuracy and loss plots confirm consistent learning.

7. The provided bar charts compare the Base CNN Model and the Improved CNN Model in terms of test accuracy and training time.

- **Test Accuracy:**
  - The base CNN Model achieved an accuracy of around 75%.
  - The Improved CNN Model showed significant improvement, achieving over 80% accuracy.
  - From this we can infer that the modifications such as hyperparameter tuning, inclusion of convolutional layer, optimization techniques enhanced the models performance
- **Training Time:**
  - The Base CNN Model took approximately 500 seconds to train.
  - The Improved CNN Model had a higher training time compared to the base CNN, indicating that additional complexity was introduced.

Visualization and Performance Insights

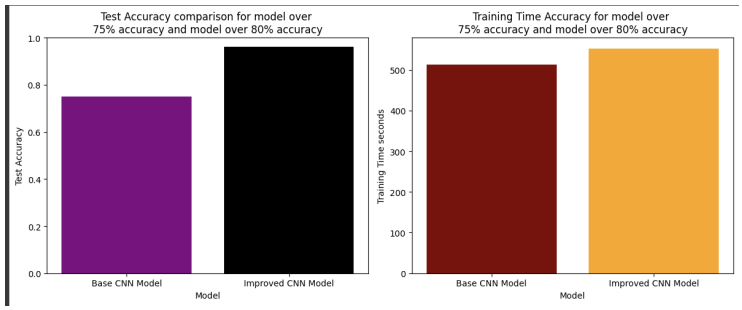


Figure 6: base Model Vs improved model visualization

- The first bar chart clearly illustrates the accuracy boost in the Improved CNN Model.
- The second chart shows that that there is slight increase in the training time, however the improvement in the test accuracy justifies the additional computational cost

PART 4 - Deep Learning Theoretical part

1.

$$\begin{aligned} h_1 &= (0.7 \times -0.3) + (0.5 \times 0.15) + (1 \times 0.9) \\ &= -0.21 + 0.075 + 0.9 \\ &= 0.765 \end{aligned}$$

$$\begin{aligned} h_2 &= (0.7 \times 0.8) + (0.5 \times 0.2) + (1 \times -0.14) \\ &= 0.56 + 0.1 - 0.14 \\ &= 0.52 \end{aligned}$$

$$\hat{y} = (h_1 \times 0.7) + (h_2 \times 0.25) + (1 \times -0.1)$$

Substituting the values:

$$\begin{aligned} \hat{y} &= (0.765 \times 0.7) + (0.52 \times 0.25) + (-0.1) \\ &= 0.5355 + 0.13 - 0.1 \\ \hat{y} &= 0.5655 \end{aligned}$$

2. MSE Calculation

$$MSE = 1/2 \times (y - \hat{y})^2$$

$$y = 0.5$$
$$\hat{y} = 0.5655$$

$$= 1/2 \times (0.5 - 0.5655)^2$$
$$= 1/2 \times (-0.0655)^2$$
$$= 0.0042/2$$
$$MSE = 0.0021$$

3. Gradient using back propagation

$$\frac{\partial \text{MSE}}{\partial \hat{y}} = (\hat{y} - y) = (0.5655 - 0.5) = 0.0655$$

$$\frac{\partial \text{MSE}}{\partial w_7} = 0.0655 \times h_1 = 0.0655 \times 0.765 = 0.0501$$

$$\frac{\partial \text{MSE}}{\partial w_8} = 0.0655 \times h_2 = 0.0655 \times 0.52 = 0.0341$$

$$\frac{\partial \text{MSE}}{\partial w_9} = 0.0655 \times 1 = 0.0655$$

$$\frac{\partial \text{MSE}}{\partial h_1} = 0.0655 \times w_7 = 0.0655 \times 0.7 = 0.04585$$

$$\frac{\partial \text{MSE}}{\partial h_2} = 0.0655 \times w_8 = 0.0655 \times 0.25 = 0.016375$$

$$\frac{\partial \text{MSE}}{\partial z_1} = 0.04585 \times (1 \text{ if } z_1 > 0) = 0.04585$$

$$\frac{\partial \text{MSE}}{\partial z_2} = 0.016375 \times (1 \text{ if } z_2 > 0) = 0.016375$$

$$\frac{\partial \text{MSE}}{\partial w_1} = 0.04585 \times x_1 = 0.04585 \times 0.7 = 0.0321$$

$$\frac{\partial \text{MSE}}{\partial w_2} = 0.04585 \times x_2 = 0.04585 \times 0.5 = 0.0229$$

$$\frac{\partial \text{MSE}}{\partial w_3} = 0.016375 \times x_1 = 0.016375 \times 0.7 = 0.01146$$

$$\frac{\partial \text{MSE}}{\partial w_4} = 0.016375 \times x_2 = 0.016375 \times 0.5 = 0.00819$$

$$\frac{\partial \text{MSE}}{\partial w_5} = 0.04585 \times 1 = 0.04585$$

$$\frac{\partial \text{MSE}}{\partial w_6} = 0.016375 \times 1 = 0.016375$$

4. Update Weights

Using learning rate $\alpha = 0.03$:

$$w_7 = 0.7 - 0.03(0.0501) = 0.6985$$
$$w_8 = 0.25 - 0.03(0.0341) = 0.249$$
$$w_9 = -0.1 - 0.03(0.0655) = -0.102$$
$$w_1 = -0.3 - 0.03(0.0321) = -0.301$$
$$w_2 = 0.8 - 0.03(0.0229) = 0.799$$
$$w_3 = 0.15 - 0.03(0.01146) = 0.1497$$
$$w_4 = 0.2 - 0.03(0.00819) = 0.1997$$
$$w_5 = 0.9 - 0.03(0.04585) = 0.8986$$
$$w_6 = -0.14 - 0.03(0.016375) = -0.1405$$

5. computation graph for the forward and backward pass

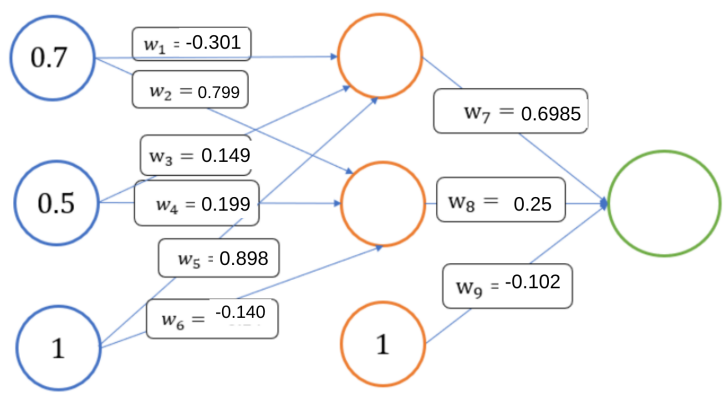


Figure 7: new wights

6. perform forward pass to estimate predicted output using the updated weights

$$\begin{aligned} h_1 &= (0.7 \times -0.301) + (0.5 \times 0.149) + (1 \times 0.898) \\ &= -0.2107 + 0.0745 + 0.898 \\ &= 0.7348 \end{aligned}$$

$$
\begin{aligned}
h_2 &= (0.7 \times 0.799) + (0.5 \times 0.199) + (1 \times -0.14) \\
&= 0.5593 + 0.0995 - 0.14 \\
&= 0.5188
\end{aligned}
$$

$= (\mathrm{h}_1 \times 0.6985) + (h_2 \times 0.25) + (1 \times -0.102)$
$= (0.7618 \ \times 0.6985) + (0.5188 \times 0.25) + (1 \times -0.102)$
$\hat{y} = 0.5598$

7. Estimating the MSE

$$MSE = 1/2 \times (y - \hat{y})^2$$

$$
\begin{aligned}
y &= 0.5 \\
\hat{y} &= 0.5598
\end{aligned}
$$

$$
\begin{aligned}
&= 1/2 \times (0.5 - 0.5598)^2 \\
&= 1/2 \times (-0.0598)^2 \\
&= 0.00357/2 \\
MSE &= 0.00178
\end{aligned}
$$

2. Derivative of Tanh

f(x) $= \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$
$f'(x) = \frac{(e^x - e^{-x})'(e^x + e^{-x}) - (e^x - e^{-x})(e^x + e^{-x})'}{(e^x + e^{-x})^2}$
$\frac{d}{dx}(e^x - e^{-x}) = e^x + e^{-x}, \quad \frac{d}{dx}(e^x + e^{-x}) = e^x - e^{-x}$
$f'(x) = \frac{(e^x + e^{-x})(e^x + e^{-x}) - (e^x - e^{-x})^2}{(e^x + e^{-x})^2}$
$(e^x + e^{-x})^2 - (e^x - e^{-x})^2$
$(e^x + e^{-x})^2 - (e^x - e^{-x})^2 = 4e^x e^{-x} = 4$
$f'(x) = \frac{4}{(e^x + e^{-x})^2} = \mathrm{sech}^2(x)$
$f'(x) = 1 - \tanh^2(x)$
$f'(x) = 1 - f(x)^2$

reference: https://pytorch.org/tutorials/beginner/introyt/introyt$_{index.html}$